

1.1 - Implementación directa de `hello_world`

Este proyecto implementa directamente en C el modelo `hello_world` usado por `esp-tflite-micro`. El modelo recibe un valor x entre 0 y 2π y estima $\sin(x)$ con tres capas densas cuantizadas en int8:

- **dense:** 1 entrada $\rightarrow$ 16 neuronas, ReLU
- **dense_1:** 16 $\rightarrow$ 16 neuronas, ReLU
- **dense_2:** 16 $\rightarrow$ 1 salida

Los pesos, sesgos y parámetros de cuantización fueron extraídos del ejemplo `esp-tflite-micro:hello_world`. La inferencia no utiliza el intérprete de TensorFlow Lite Micro.

Compilar y ejecutar

```
cd Lab_3/1.1
```

```
idf.py set-target esp32
```

```
idf.py build
```

```
idf.py -p /dev/ttyUSB0 flash monitor
```

La salida por serial queda en formato CSV:

```
source,x,y_model,y_sin,avg_inference_us,q_input,q_output
```

```
direct,0.000000,0.050832,0.000000,12.340,-128,10
```

`avg_inference_us` corresponde al tiempo promedio de una inferencia directa, calculado con `esp_timer_get_time()` sobre `BENCHMARK_REPEATS` repeticiones.

Medición de potencia

Para medir potencia durante la inferencia, editar `main/lab3_config.h` y cambiar `MEASURE_GPIO` desde `-1` a un pin libre. El firmware pondrá ese pin en alto únicamente durante la ventana de benchmark. Con un power profiler, osciloscopio o analizador lógico se puede sincronizar el consumo de corriente con dicha ventana.

La potencia se obtiene mediante:

$$P = V \times I$$

Posteriormente, comparar contra el ejemplo de TensorFlow Lite Micro ejecutado bajo las mismas condiciones de placa, alimentación y frecuencia.

Comparar con TensorFlow Lite Micro

Ejecutar el ejemplo base y guardar su monitor:

```
cd ~/esp/projects_tf/hello_world
```

```
idf.py -p /dev/ttyUSB0 flash monitor | tee tfmicro.log
```

Ejecutar esta implementación directa y guardar su monitor:

```
cd Lab_3/1.1
```

```
idf.py -p /dev/ttyUSB0 flash monitor | tee direct.log
```

Comparar salidas:

```
python3 tools/compare_outputs.py direct.log tfmicro.log
```

El script genera `comparison.csv` y, si `matplotlib` está instalado, `comparison.png`. Si `matplotlib` no está instalado, genera `comparison.svg`.

Resultados capturados

Con el ESP32-CAM conectado en `/dev/ttyUSB0` se flashearón ambos firmwares y se guardaron los siguientes archivos:

- `direct.log`: salida serial de esta implementación directa.
- `tfmicro.log`: salida serial del ejemplo con TensorFlow Lite Micro.
- `comparison.csv`: comparación punto a punto y diferencia de tiempo.
- `comparison.svg`: visualización de $\sin(x)$, `direct C` y `TF Micro`.
- `power_measurements.csv`: mediciones de voltaje, corriente y potencia obtenidas con el medidor USB.
- `direct_power_measurement.png`: fotografía de la medición de potencia para `direct`.
- `tfmicro_power_measurement.png`: fotografía de la medición de potencia para `TF Micro`.

Resumen de tiempo promedio de inferencia

direct: 73.928 us

TF Micro: 61.463 us

direct - TF Micro: 12.465 us

Resumen de potencia medida

direct: $5.1018 \text{ V} \times 0.11205 \text{ A} = 0.5717 \text{ W}$ (display: 0.5714 W)

TF Micro: $5.1083 \text{ V} \times 0.09782 \text{ A} = 0.4997 \text{ W}$ (display: 0.5000 W)

direct - TF Micro: 0.0720 W

Las fotografías de respaldo quedaron almacenadas como `direct_power_measurement.png` y `tfmicro_power_measurement.png`.